

Sistemi Digitali

25/09 – Lezione Introduttiva

È il corso preliminare di architettura degli elaboratori del secondo semestre e studiamo come è costruito e come funziona un computer. Nella 1° parte studieremo solo la progettazione di sistemi NON dotati di memoria:

- Rappresentazione delle informazioni
- Porte logiche e circuiti
- Algebra di boole come supporto matematico per progettare in maniera efficiente
- Moduli standard come codificatori, decodificatori, multiplexer, rom etc.

Nella 2° parte del corso apprenderemo:

- Cellule di memorizzazione e registri
- Progettazione di circuiti sequenziali
- Automi a stati finiti
- Registri Standard come registri di memorizzazione o contatore
- Hardware Description Language (nel nostro caso Verylog)

L'Architettura di Von Neuman

L'Architettura di Von Neumann (che partecipò alla creazione dei primi computer) fu la prima architettura a programma memorizzato ed era composta da un processore (CPU), una Memoria e Input/Output. Questi elementi erano collegati tra loro tramite bus. E si svolge in diverse funzioni:

- elaborazione
- memorizzazione
- scambio di informazioni
- controllo (svolto dalla CPU)

Rappresentazione dell'Informazione

Per la rappresentazione è necessario:

- un alfabeto di supporto e parole
- sequenze finite di simboli alfabetici

Funzione di codifica: una funzione che associa informazioni nell'insieme 1 a parole nell'insieme 2 detto codice. Abbiamo quindi un insieme detto Dominio che arriva ad un altro insieme, detto Codominio.

La Cardinalità è il numero finito di elementi contenuti in un insieme

NI: Cardinalità di Informazioni

NC: Cardinalità di Codice

Se $NI < NC$ parliamo di **codifica ridondante**

Se $NI > NC$ non si hanno sufficienti parole di codice per rappresentare tutte le informazioni ed avremo quindi una **codifica ambigua**

L'ideale sarebbe avere lo stesso numero di informazioni e di parole di codice, per cui ad ogni informazione corrisponde una e una sola parola di codice.

Esistono inoltre dei requisiti di rappresentazione. E sono:

- **Economicità:** utilizzare il minor numero possibile di caratteri per rappresentare le informazioni
- **Semplicità di Codifica e Decodifica** al fine di rendere più semplice possibile la codifica e la decodifica
- **Semplicità di Elaborazione**

Per la rappresentazione delle informazioni solitamente si utilizzano numeri naturali in base 10, ma possono essere adottati anche criteri diversi come la Base 2, Base 8, Base 16.

Possiamo quindi utilizzare una base “x” qualsiasi raggruppate in “n” cifre. Utilizzeremmo “x” simboli diversi {0, 1, 2, 3, ..., b-1} con un intervallo di [0; bⁿ-1]. Per esempio con 4 cifre si potranno rappresentare 10⁴ valori con un intervallo di [0; 10⁴-1]

In questo caso la base rappresenta il nostro alfabeto e le “n” cifre la lunghezza delle nostre parole

Base 10: (Decimale → Decimal)

Utilizza 10 simboli diversi {0;9}

Base 2: (Binario → Binary)

Utilizza solo 2 simboli diversi {0;1}

Base 8: (Ottale → Octal)

Utilizza i simboli {0; 7} raggruppati in 3 Bit ($\log_2 8$)

Base 16: (Esadecimale → Hexadecimal)

Utilizza i simboli {0;9, a, b, c, d, e, f} perché da 9 in poi si utilizzano le lettere, raggruppati in 4 Bit ($\log_2 16$)

Conversione

Per la Conversione da base 10 a base b dato N:

- Dividere N per b
- Registriamo il resto
- Iterare la divisione sul quoziente e registrare il resto fino a quoziente = 0
- La sequenza di resti, in ordine inverso, dà la rappresentazione richiesta (rappr. In base b di N)

Esempio conversione base 10 a Binario:

1203		0
601		0
300		1
150		1
75		0
37		0
18		1
9		1
4		0
2		1
1		0
0		0

$$1203 = 001011001100$$

Per la Conversione da base b a base 10:

$$1011 \text{ da Binario a Decimale} \rightarrow \begin{array}{c|c|c|c} 2^3 & 2^2 & 2^1 & 2^0 \\ \hline 1 & 0 & 1 & 1 \end{array} \rightarrow 1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 = 11$$

(operazione inversa della potenza)

Per capire di quanti bit ho bisogno prendiamo la parte intera superiore del Logaritmo in base 2 dell'alfabeto che vogliamo utilizzare. Per esempio:

$$\text{ottale} \rightarrow n. \text{ bit} = \log_2 8$$

$$\text{esadecimale} \rightarrow n. = \log_2 16$$

{ } Rappresentano un insieme

[] Rappresenta un intervallo finito di valori

29/09

$$\text{KiloByte} \rightarrow 2^{10} = 1024 \rightarrow 10^3 = 1\,000$$

$$\text{MegaByte} \rightarrow 2^{20} = \dots \rightarrow 10^4 = 10\,000$$

$$\text{GigaByte} \rightarrow 2^{30} = \dots \rightarrow 10^5 = 100\,000$$

Addizioni, Sottrazioni e Moltiplicazioni in basi diverse da 10

Sul quaderno

Rappresentazione dei Numeri Interi (Z)

Rappresentare i Numeri Negativi risulta complicato perché tutte le possibili combinazioni (2^n) sono già state utilizzate per la rappresentazione dei numeri positivi. Tuttavia sono stati trovati dei metodi che però, dimezzano il valore massimo che possiamo raggiungere. Tali metodi sono:

- **Rappresentazione in modulo e segno**

il primo bit ci indica il segno: 0 = Segno Positivo, 1 = Segno Negativo

Rende semplice il cambio di segno, perché basta cambiare l'ultima cifra, ma crea dei problemi per svolgere le ulteriori operazioni aritmetiche.

- **Rappresentazione in complemento a 1**

Il negativo di un numero si ottiene scambiando gli 0 e gli 1.

- **Rappresentazione in complemento a 2** (anche detta complemento alla Base)

È la più utilizzata e assegna peso negativo al MostSignificantBit (MSB, quello più a sinistra). Posso rappresentare $[-2^{n-1}; 2^{n-1}-1]$ valori ed è il metodo più utilizzato perché permette di calcolare abbastanza facilmente l'opposto, ed allo stesso tempo permette di svolgere con facilità addizioni e sottrazioni a patto che siano rappresentabili.

Per calcolare l'opposto complemento tutti i bit (inverto 0 e 1) e sommo 1.

Per estendere le rappresentazioni di numeri positivi basta aggiungere zeri in testa. Per i numeri negativi basterà mettere tutti uno, e manterrò lo stesso valore secondo una proprietà delle potenze di 2.

02/10

Di quanti bit ho bisogno per rappresentare un determinato numero sia in positivo che in negativo?

- Per MS basta aggiungere un bit a sx con 0 o 1 per la rappresentazione del segno.
- Per CA1 sono sufficienti lo stesso numero di bit, perché basta complementarlo interamente.
- Per CA2 basta aggiungere un bit per raddoppiare l'intervallo che posso rappresentare.

Rappresentazione dei numeri Reali (R)

Conversione dei numeri con la virgola

Si procede separatamente per la parte intera e quella decimale. La parte intera già la sappiamo fare. Per la parte decimale si utilizza il sistema posizionale tenendo conto che dietro la virgola si hanno esponenti negativi. Si moltiplica la parte decimale (0,75) per la base e si ripete lo stesso procedimento anche sui decimali del numero ottenuto dalla moltiplicazione fino a che arriviamo ad avere zero o raggiungiamo il numero di bit massimo che adottiamo nella nostra rappresentazione.

$10^1 10^0 10^{-1} 10^{-2}$

Es. 13,75

$$13 = 1101$$

$$0,75 \times 2 = \boxed{1}5 \rightarrow 1101,11 = 13,75$$

$$0,50 \times 2 = \boxed{1}$$

Altro es. = 13,45

$$13 = 1101$$

$$0,45 \times 2 = \boxed{0}9$$

$$0,90 \times 2 = \boxed{1}8$$

$$0,80 \times 2 = \boxed{1}6 \rightarrow 1101,011100 = 13,45$$

$$0,60 \times 2 = \boxed{1}2$$

$$0,20 \times 2 = \boxed{0}4$$

$$0,40 \times 2 = \boxed{0}8 \text{ e da qui in poi le cifre iniziano a ripetersi}$$

Rappresentazione in Virgola Fissa

Decido fin dal principio quanti bit utilizzare per la parte intera e quanti per la parte decimale. Dunque:

- n è la lunghezza totale
- k i bit disponibili per la parte intera
- n-k i bit rimasti disponibili per la parte decimale.

Tuttavia questo metodo risulta parecchio scomodo in alcuni casi ed è quindi poco utilizzato.

Rappresentazione in Virgola Mobile (floating point)

Per la rappresentazione in virgola mobile utilizzeremo una tripla di valori:

- Un solo bit per il segno (0 per i positivi e 1 per i negativi)
 - N. bit per l'Esponente
 - N. bit per la Mantissa (la parte che troviamo dopo la virgola)
- } Il n. di Bit dedicati dipenderà dallo standard che utilizzi.

Lo standard più utilizzato per la virgola mobile è l'IEEE754, e divide i bit nel seguente modo:

- 16 Bit → Half Precision → e = 5bit m = 10bit bias = 15
- 32 Bit → Single Precision → e = 8bit m = 23bit bias = 127
- 64 Bit → Double Precision → e = 11bit m = 52bit bias = 1023

Il valore associato alla tripla si calcola facendo → $(-1)^s \times 1, m \times 2^e$

E l'Intervallo per l'esponente sarà → $[-2^{e-1}, 2^{e-1} - 1]$

- Si eliminano i due valori più piccoli (cioè -2^{e-1} e $-2^{e-1} + 1$) e si considera $[-2^e + 2, 2^{e-1} - 1]$
- Si somma il BIAS (cioè $2^{e-1} - 1$) e otteniamo $[1, 2^e - 2]$

Svolgimento

Sposto la virgola fino all'1 più a sinistra (in questo caso di 3 posizioni) per portarlo in notazione scientifica. Dovrò quindi moltiplicare il valore per 2^3 . Perciò, la rappresentazione in virgola mobile sarà:

0 (per il segno positivo)

0011 (cioè 3 che è l'esponente espresso in binario) → 0 0011,1010111000

1010111000 (la mantissa, che scriverò finché ho bit a disposizione)

IEEE754 a 16Bit

Convertire 13,45

1101, 011100 – lo porto in notazione scientifica → $1,101011100 \times 2^3$ ← Sposto la virgola di 3 posizioni quindi e = 3

$e = 3 + 15 = 18 \rightarrow 10010$

Esponente bias

E quindi il risultato sarà: 0 10010 1010111000 – che raggruppati in 4bit e portati in base 16 fa → 4AB8

Il segno L'Esponente La Mantissa
della not. scientifica

06/10

Addizioni e Sottrazioni in virgola mobile (esempio sul quaderno)

- Confrontare gli esponenti e portarli allo stesso valore (solitamente al più grande dei due). Questo equivale a modificare una delle due mantisse.
- Spostare a destra la mantissa per mantenere il valore dell'addendo a cui stiamo modificando l'esponente.
- Eseguire addizione o sottrazione a seconda del segno degli addendi, scegliendo l'ordine degli addendi e il segno del risultato coerentemente.

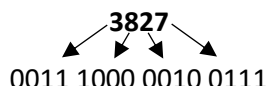
Può verificarsi l'overflow se l'esponente diventa troppo grande o troppo piccolo per rappresentare il risultato.

Moltiplicazione e Divisione in virgola mobile (esempio sul quaderno)

- Sommare/sottrarre gli esponenti
- Moltiplicare/dividere le mantisse.
- Segno negativo se ho operandi discordi o positivo con operandi concordi
- Il risultato può dover essere normalizzato (il che equivale a portare la mantissa nella forma 1,aaaa e regolare l'esponente di conseguenza).

BCD → Binary Coded Decimal

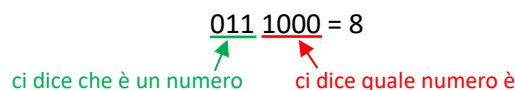
Codifica ogni singola cifra. Per ogni cifra avrò bisogno di 4 bit.



Quello che utilizziamo noi è il BCD Standard, che sfrutta le potenze di 2, ma esistono anche altri BCD più complessi.

ASCII → American Standard Code for Information Interchange

Il codice ASCII è il codice binario standard per la rappresentazione di caratteri alfanumerici. Esso permette quindi di rappresentare non solo numeri, ma anche caratteri e simboli. Tale codice utilizza 7 bit di cui 3 sono per il prefisso (che ci dice se si tratta di un numero, un carattere o un simbolo) e 4 per la rappresentazione (ci dice di quale numero/carattere/simbolo si tratta).



Bit di Parità

Il Bit di parità si utilizza per controllare se la trasmissione della informazione è stata corretta o presenta degli errori. Potremmo avere un Bit di parità Pari o Dispari.

- **Pari:** Si aggiunge un bit 1 per far sì che il numero di 1 nella rappresentazione sia pari.
- **Dispari:** Si aggiunge un bit 0 per far sì che il numero di 1 nella rappresentazione sia dispari.

Supponiamo di utilizzare la parità pari. Al momento della ricezione il codice viene ricontrollato e se il numero di 1 contenuti nel codice non fosse pari almeno un bit avrà cambiato valore durante la trasmissione. Ovviamente questo metodo sarà in grado di identificare solo un numero dispari di errori, e viceversa per il sistema di parità dispari.

Dopo aver individuato se c'è un errore o meno si procede con la ritrasmissione del carattere nell'ipotesi che l'errore si sia verificato casualmente oppure no. Se il ricevitore dovesse rilevare un errore di parità invierà il carattere di controllo denominato NAK (Negative Acknowledgement) al trasmettitore. In assenza di errori il ricevitore invierà invece il carattere ACK (Acknowledgement).

Se dopo un certo numero di tentativi gli errori di trasmissione dovessero continuare a corrompere il messaggio, può essere inviato all'operatore un messaggio per segnalare la presenza di errori sulla linea di trasmissione.

Logica di Boole

La Logica Binaria, anche detta logica di Boole o Booleana, tratta variabili che possono assumere solo due valori, 1 e 0, e le operazioni di logica matematica che si possono svolgere con tali variabili. Le tre principali e fondamentali operazioni logiche sono AND, OR e NOT:

1. **AND** → È rappresentata da un punto o dall'assenza dell'operatore. Per esempio $Z = XY$ (si legge "Z è uguale a X AND Y"). Il risultato sarà $Z=1$ se e solo se $X=1$ E $Y=1$. Altrimenti $Z=0$.
2. **OR** → È rappresentata dal simbolo dell'addizione. Per esempio $Z=X+Y$ (si legge "Z è uguale a X OR Y"). Il risultato sarà $Z=1$ se $X=1$ O $Y=1$. Altrimenti $Z=0$ (Siccome anche $X=0$ e $Y=0$).
3. **NOT** → È rappresentato da un tratto sopra la variabile ed è anche detta complementazione. Per esempio $Z=X$ (si legge "Z è uguale a X NEGATO"). Il risultato sarà $Z=1$ se $X=0$. Oppure $Z=0$ se $X=1$.

TABELLA 2.1 Tabella di verità per le tre operazioni logiche fondamentali

AND			OR			NOT	
X	Y	$Z = X \cdot Y$	X	Y	$Z = X + Y$	X	$Z = \bar{X}$
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

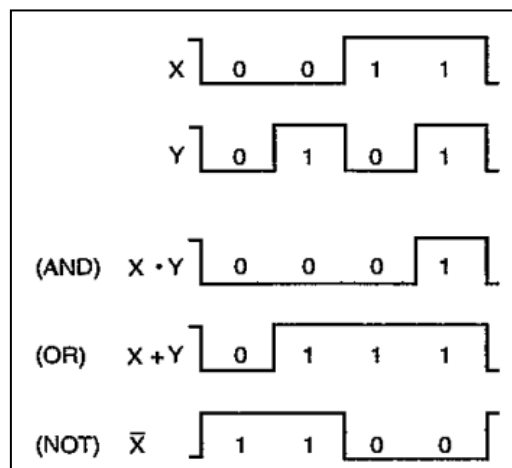
Tuttavia la Logica Binaria non va assolutamente confusa con l'Aritmetica Binaria. Infatti, mentre l'aritmetica binaria può assumere un qualsiasi valore in base 2, una variabile logica può assumere soltanto valori 1 o 0

Porte Logiche

Le Porte Logiche sono circuiti elettronici che operano su uno o più segnali di ingresso per produrre un segnale di uscita, che potrà essere uguale ad 1 o 0 logico. Di seguito le porte logiche, con rispettivi dati di ingresso e risultati, delle principali operazioni della Logica Binaria:



Diagrammi Temporal



Algebra Booleana

L'Algebra Booleana è una struttura matematica ed ha bisogno di un alfabeto di supporto (nel nostro caso 0,1), di operatori (AND, OR, NOT) e un insieme di regole. L'algebra booleana tratta le variabili binarie e le loro relative operazioni logiche. Una funzione booleana può essere rappresentata mediante una tabella di verità che, per una data funzione, è costituita da due parti. Nella parte sinistra vengono riportate tutte le combinazioni che possono essere assegnate alle variabili binarie indipendenti, mentre nella parte destra vengono riportati i valori assunti dalla funzione in ciascuna di queste possibili combinazioni, e parliamo quindi di variabili dipendenti. Il numero di righe in una tabella di verità è pari a 2^n , essendo n il numero di variabili.

TABELLA 2.1 Tabella di verità per le tre operazioni logiche fondamentali

AND			OR			NOT	
X	Y	Z = X Y	X	Y	Z = X + Y	X	Z = $\bar{X}$
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Assiomi e Proprietà

	Somma	Prodotto
Associatività	$x + (y+z) = (x + y) + z$	$x(yz) = (xy)z$
Commutatività	$x+y = y+x$	$xy = x*y$
Distributività	$x(y+z) = xy+xz$	$x+yz = (x+y)(x+z)$
Complemento	$x + \bar{x} = 1$	$x * \bar{x} = 0$
Elemento Neutro	$x + 0 = x$	$x * 1 = x$
Elemento Nullif	$x + 1 = 1$	$x + 0 = 0$
Involuzione	$\bar{\bar{x}} = x$	
Idempotenza	$x + x = x$	$x * x = x$
Assorbimento	$x + xy = x$	$x(x+y) = x$
Legge di Demorgan	$\overline{x+y} = \bar{x} * \bar{y}$	$\overline{x*y} = \bar{x} + \bar{y}$

09/10/23

Per la verifica di identità di una (funzione o espressione?) possiamo utilizzare il **Metodo dell'induzione perfetta**. Consiste nella costruzione della tavola di verità delle due funzioni di cui vogliamo verificare l'identità. **Le funzioni potranno anche essere divise in Duali o Complementare:**

- **Una Funzione è la Duale** di un'altra se differisce per gli operatori (addizioni e moltiplicazioni scambiate) e per il valore delle costanti (0 e 1 scambiati). Passare all'espressione duale di un'identità può facilitare la verifica di identità o permetterci di creare circuiti con meno porte logiche o "spendendo di meno".
- **Una Funzione Complementare** si ottiene complementando tutti di elementi di un'altra funzione (operatori compresi. Infatti molto spesso utilizzeremo la legge di Demorgan per convertire + in * e viceversa). Si può anche ottenere, più semplicemente, svolgendo la funzione duale e complementando i valori di tutte le variabili.

In generale le funzioni vengono scritte e rappresentate mediante l'utilizzo di espressioni booleane. Queste potranno essere in forma:

- **SOP:** Sum of Product (Forma Normale Disgiuntiva) $\rightarrow xy + xz \rightarrow$ Rete AND-to-OR
- **POS:** Product of Sums (Forma Normale Congiuntiva) $\rightarrow (x+y)(x+z) \rightarrow$ Rete OR-to-AND

Come facciamo ad ottenere un'espressione in Forma Normale? Procedimento:

1. **Applicazione delle Leggi di Demorgan**, fino ad avere la complementazione sulle singole variabili.
2. **Proprietà Distributiva**, $(xy+xz)$ per le SOP e $(x+y)(x+z)$ per le POS
3. **Eliminare i Termini Ripetitivi o Ridondanti** (con assorbimento, involuzione etc.)

10/10/23

Oggi vediamo le espressioni in Forma Canonica cioè con tutti i termini MinTermine o MaxTermine (rispettivamente per SOP e POS).

- **MinTermine:** Termine prodotto in cui compaiono tutti i letterali. $\rightarrow$ SOP
- **MaxTermine:** Termine somma in cui compaiono tutti i letterali. $\rightarrow$ POS

Ogni termine in forma canonica corrisponde a un solo valore nella tavola di verità. E viceversa, per cui dalla tavola della verità posso ricavare l'espressione in forma canonica. Copiare appunti dal quad

Come facciamo ad ottenere un'espressione in Forma Canonica SOP-POS? Procedimento:

1. **La Porto in Forma Normale**
2. **SOP $\rightarrow$ Moltiplico** il Termine Prodotto in cui manca x per $(x+\bar{x}) \rightarrow ab + bc \rightarrow ab(c+\bar{c}) + ca(b+\bar{b})$
3. **POS $\rightarrow$ Sommo** il termine in cui manca x il prodotto $(x*\bar{x}) \rightarrow (a+b)(b+c) \rightarrow$
4. **Applicare la proprietà distributiva**
5. **Eliminare i Termini Ripetitivi o Ridondanti**

Circuito Minimale

- Minor numero di porte
- A parità di numero di porte, minor numero di ingressi per le porte

Espressione Minimale

- Minor numero di termini
- A parità di termini minor numero di letterali nell'espressione

Mappe di Karnaugh o k-mappe

La mappa di karnaugh redistribuisce i valori della tavola di verità in una tabella a due entrate e ci permette di ricavare espressioni minimali. Per gli esempi ed il procedimento nello specifico guardare sul quaderno. Per la costruzione dell'espressione minimale è fondamentale ricordare che:

in SOP $\rightarrow \bar{x} = 0 \quad x = 1$

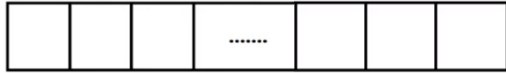
In POS $\rightarrow \bar{x} = 1 \quad x = 0$

Registri

I registri non sono altro che un insieme di flip flop, uno affianco all'altro, che vengono utilizzati per svolgere principalmente 2 funzioni:

- **Caricamento:** quando inseriamo valori all'interno di un registro
- **Memorizzazione:** per il mantenimento delle informazioni nel tempo

Rappresentazione

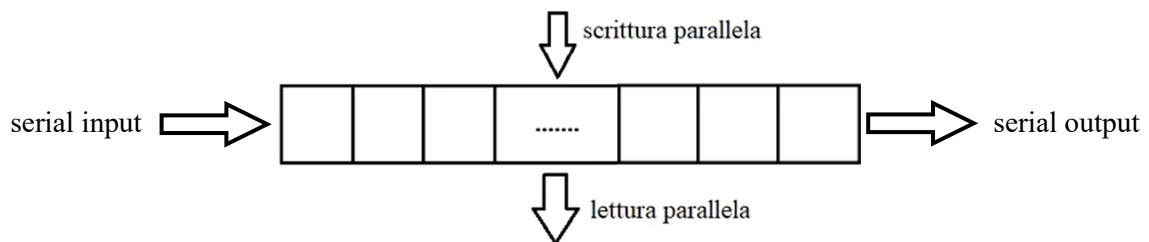


Vengono rappresentati come un insieme di celle contenenti valori, in cui ogni cella è un flip flop. Solitamente si utilizza FF D, ma possono essere utilizzati anche FF SR o JK, collegando le due entrate ad uno stesso valore.

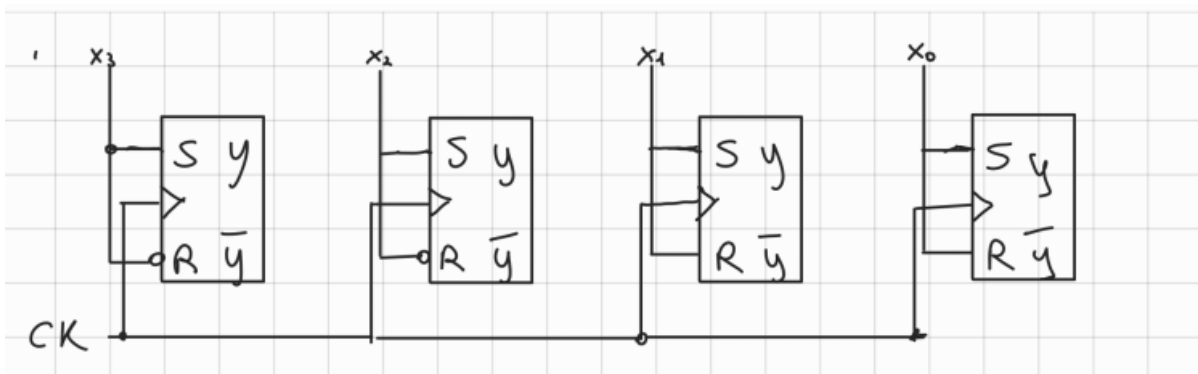
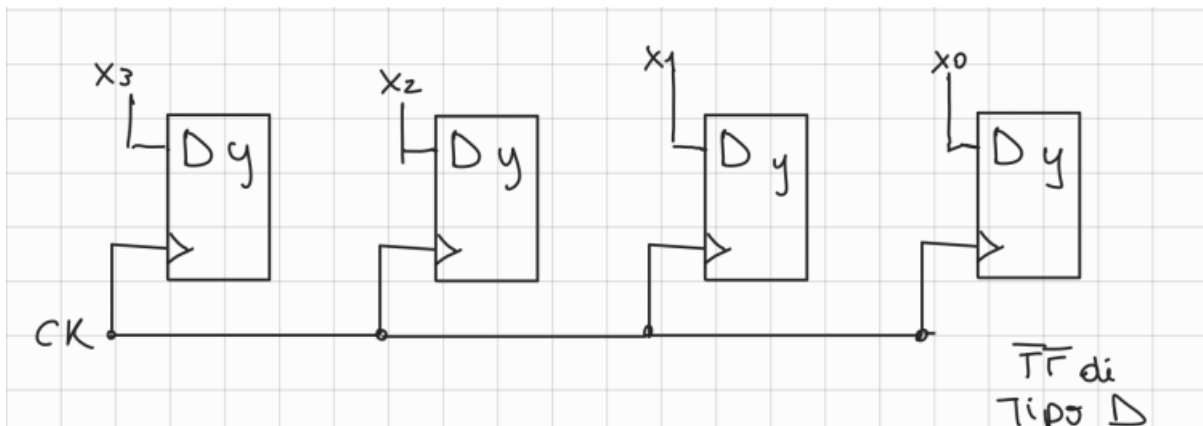
Caricamento

Il Caricamento dei Registri può avvenire in 2 modi:

- **Caricamento Parallelo:** se i dati vengono caricati tutti simultaneamente
- **Caricamento Seriale:** cioè 'in serie', dal momento che i dati vengono inseriti uno alla volta e poi trasmesso ai FF successivi svolgendo uno scorrimento.



Caricamento Parallelo



Distinzione Caricamento/Memorizzazione

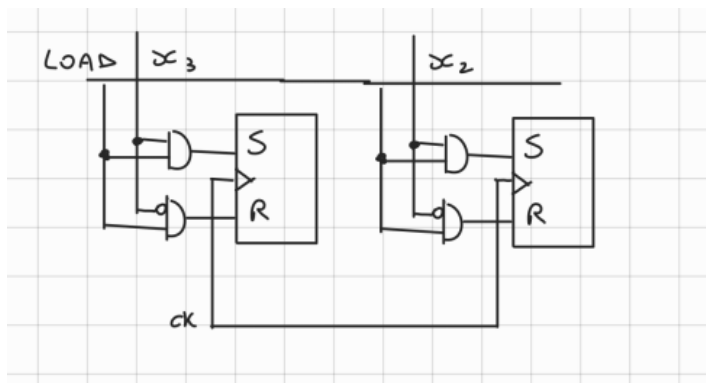
Per decidere invece di svolgere l'altra funzione dei registri, cioè la memorizzazione e mantenimento delle informazioni, possiamo decidere di aggiungere una nuova linea di controllo, chiama Load.

- Quando $LOAD = 1$ andremo ad effettuare CARICAMENTO.
- Quando $LOAD = 0$ andremo ad effettuare MEMORIZZAZIONE.
-

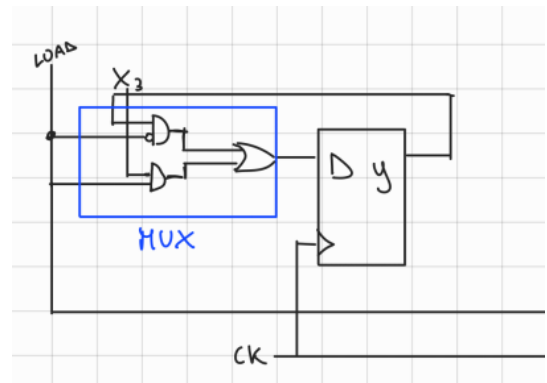
Questa linea di controllo la possiamo integrare in 2 modi:

- **Possiamo metterla in AND con gli ingressi**, cosicché se $LOAD = 0$:
 - Nel FF SR avremo $S=0$ e $R=0$ (cioè proprio Memorizzazione).
 - Nel FF D creiamo un MUX che se $Load = 0$ reinserisce nel FF lo stato precedente. Altrimenti acquisisce il nuovo valore.

FF SR

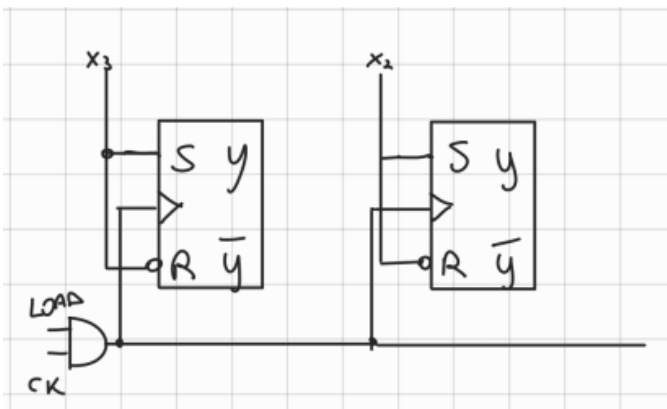


FF D

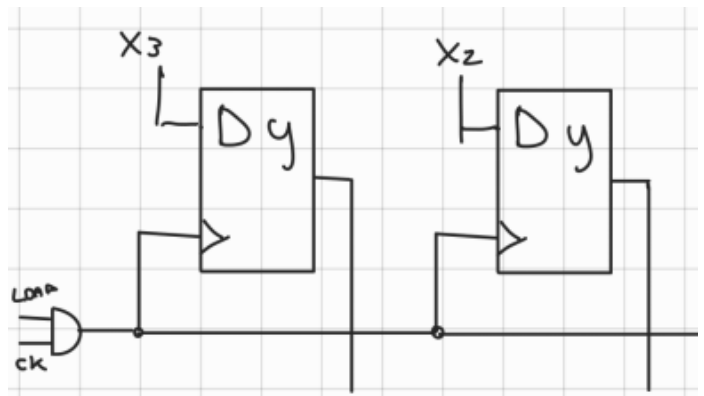


- **Possiamo agire sul Clock**, cosicché le informazioni vengano mantenute finché sia Clock che Load sono contemporaneamente uguali a 1 (cioè se abbiamo la funziona di Caricamento). *Per fare ciò basta semplicemente mettere in AND Clock e Load*

FF SR



FF D

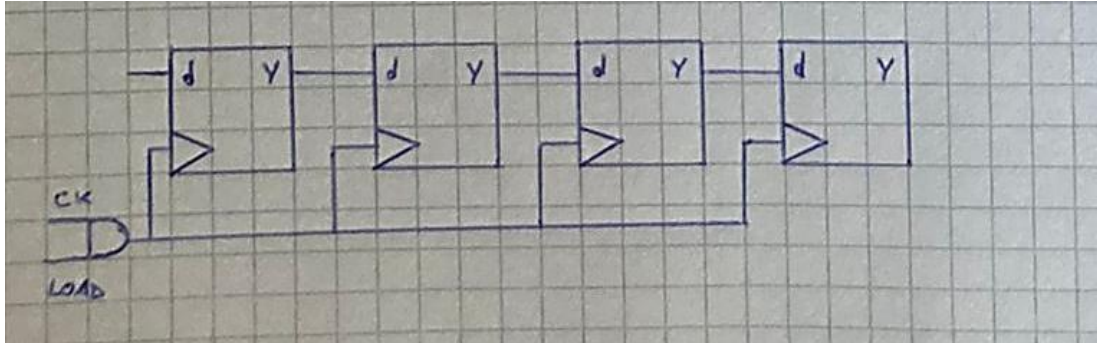


Caricamento Seriale

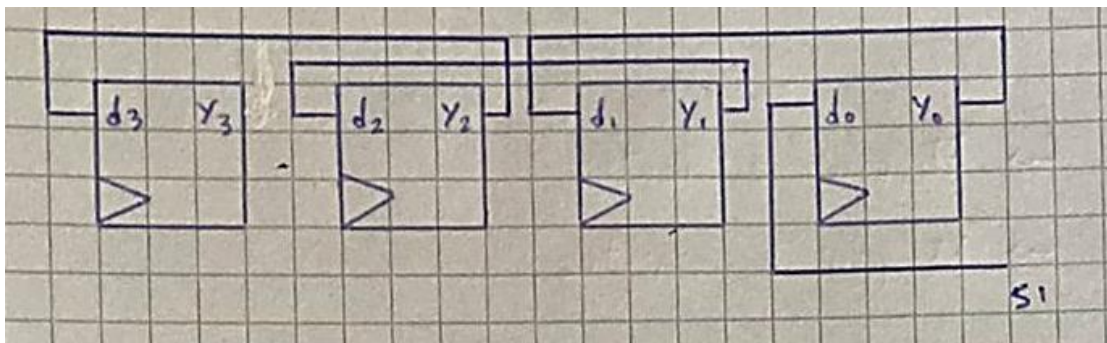
Per il Caricamento Seriale, come detto prima, i dati vengono inseriti nei FF uno alla volta, ad ogni colpo di clock, e poi passati al FF successivo. Infatti secondo questo ragionamento si sviluppa una sorta di scorrimento dei valori tra una cella e l'altra del Registro, che proprio per questo motivo ci porta a chiamarli "Shift Register".

Si noti che i disegni seguenti hanno già implementato il segnale di controllo Load, che funziona alla stessa identica maniera del caricamento parallelo, permettendo di decidere se effettuare caricamento o memorizzazione.

Shift a Destra (Il serial input viene caricato dal ff più a sinistra e passato a mano a mano al ff alla sua destra)

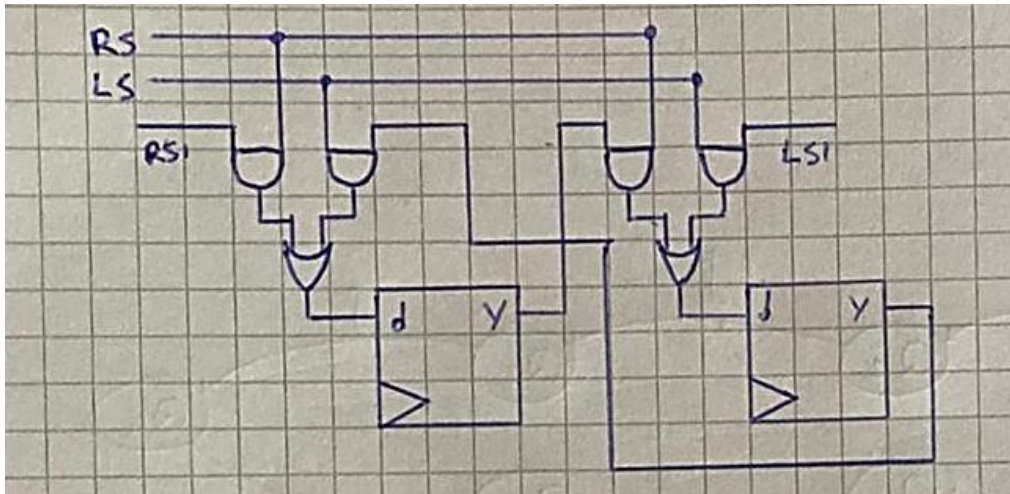


Shift a Sinistra (Il serial input viene caricato dal ff più a destra e passato a mano a mano al ff alla sua sinistra)



Shift Register Bidirezionale

Questa possibilità di scorrere i valori a destra o a sinistra comporta inevitabilmente la nascita di un nuovo tipo di registro a scorrimento. Quello in grado di cambiare lo scorrimento dei dati a nostro piacimento. Ancora utilizziamo delle linee di controllo. In particolare **RS → Right Shift** (per lo scorrimento a Destra) e **LS → Left Shift** (per lo scorrimento a Sinistra)

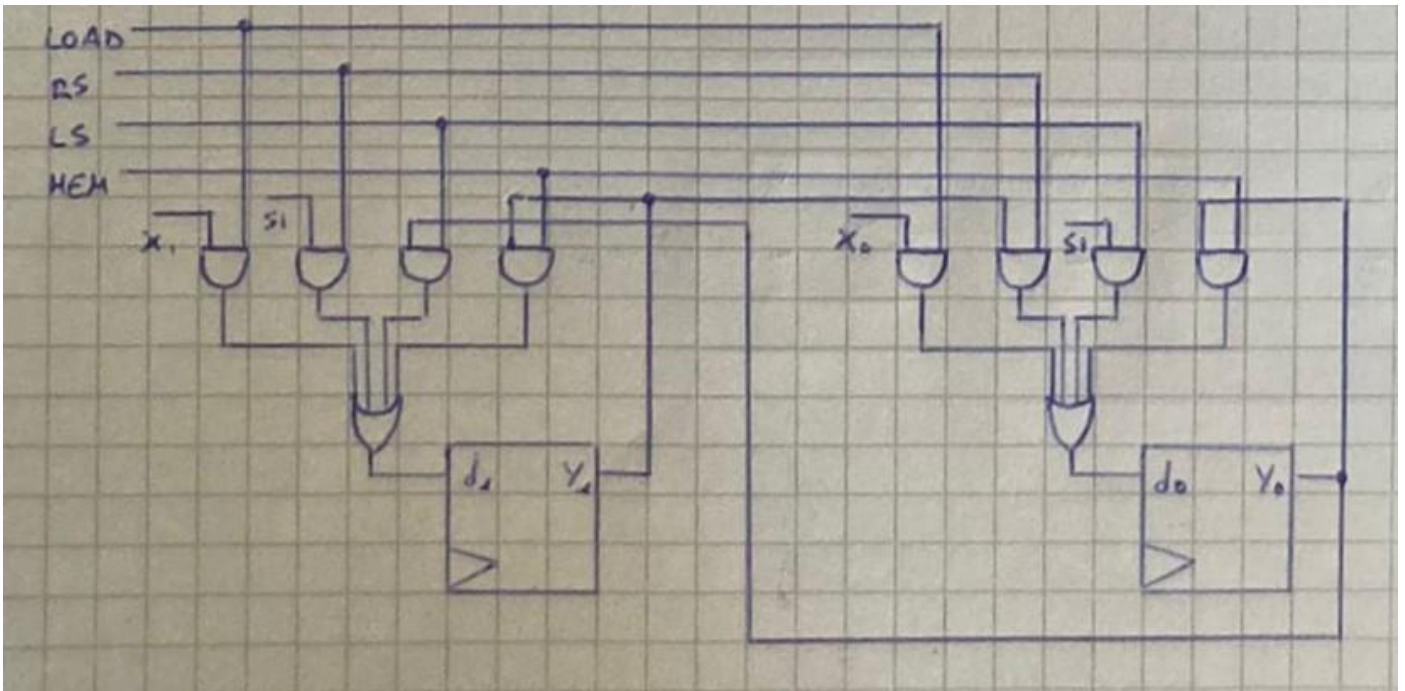


In realtà sono abbastanza sicuro che si possa realizzare anche inserendo una sola linea di controllo che se vale 1 esegue shift a destra mentre se vale 0 esegue uno shift a sinistra. Il tutto darebbe vita ad un MUX 2:1

Universal Shift Register

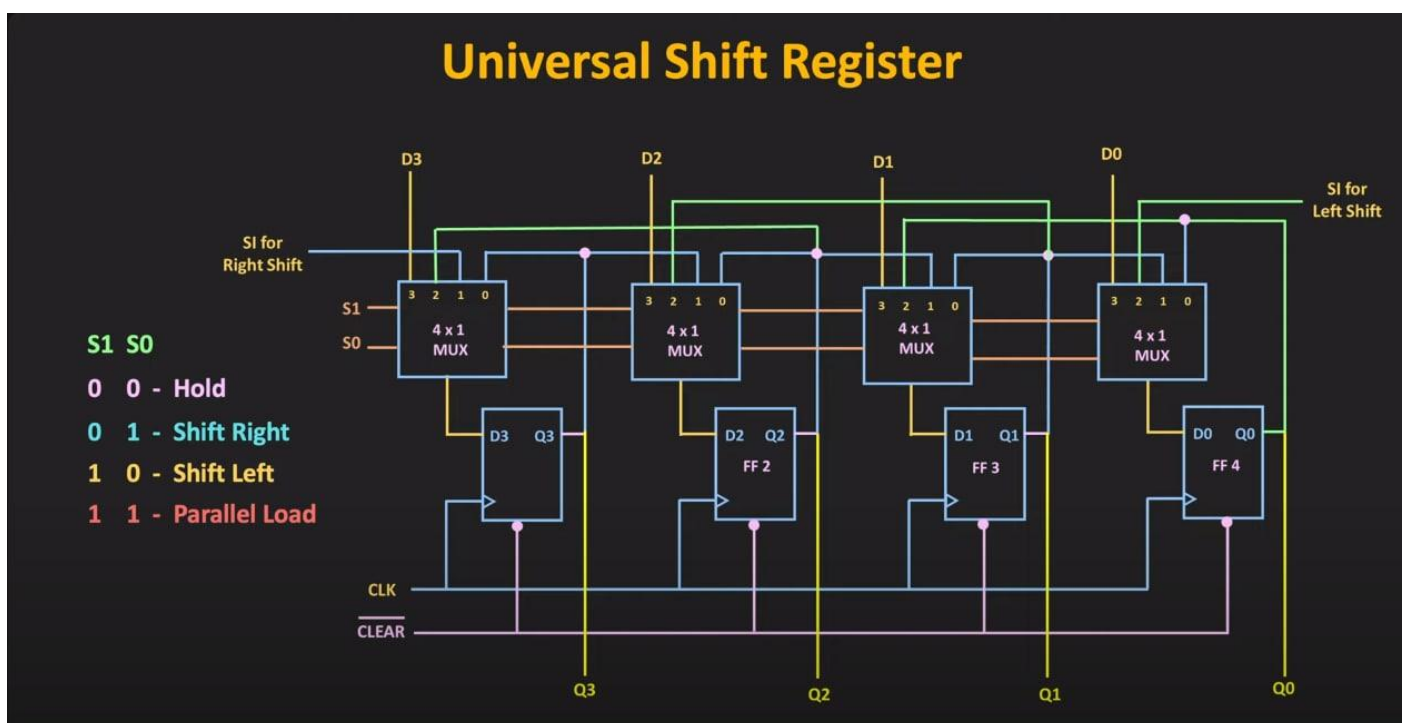
L'Universal Shift Register, anche detto Registro Universale, è un particolare Shift Register che unisce tutte le funzioni viste fino ad ora. Si hanno ben 4 linee di controllo:

- **Load:** Permette di effettuare il Caricamento Parallelo a tutti i FF
- **Mem:** Per effettuare la Memorizzazione (cioè il mantenimento dell'informazione allo stato precedente)
- **RS (Right Shift):** Per il Caricamento Seriale con scorrimento a Destra
- **LS (Left Shift):** Per il Caricamento Seriale con scorrimento a Sinistra



Rappresentazione con MUX 4:1

Osservando attentamente si può notare come gli ingressi dei FF vengano in realtà gestiti attraverso MUX 4:1 che utilizzano 2 bit di controllo. Di seguito la rappresentazione:



Registri Contatori

I Contatori sono un particolare tipo di Registri che, in base al numero di FF utilizzati, ci consentono di contare fino a un certo numero. In particolare se ho un contatore di n FF, sarò in grado di contare fino a $2^n - 1$, e viene detto contatore MODULO 2,4,8,16 etc (in base a quanto posso contare). Per esempio se ho un contatore con 3 FF, viene detto Contatore MODULO 8 ($2^3 = 8$) e ci permette di contare da 0 a 7.

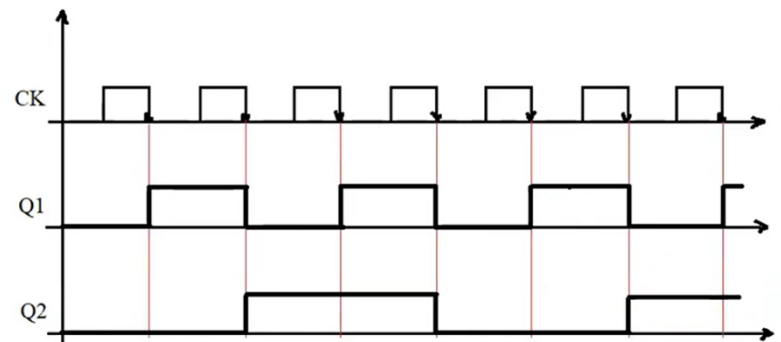
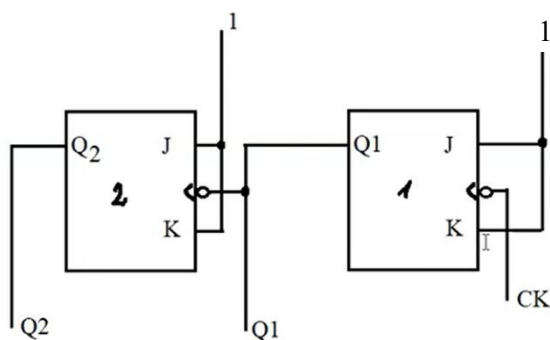
In generale i contatori utilizzano FF di tipo JK impostati su Toggle, cioè con entrambi gli ingressi uguali a 1. In questo modo ad ogni colpo di clock il valore viene complementato creando una sequenza del tipo 010101 (se utilizzassi 1 FF solo).

Contatori Sincroni/Asincroni

Un importante distinzione per i Contatori è quella tra:

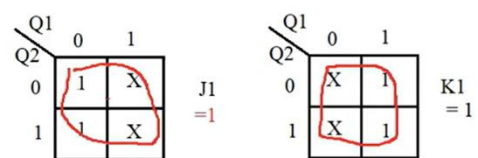
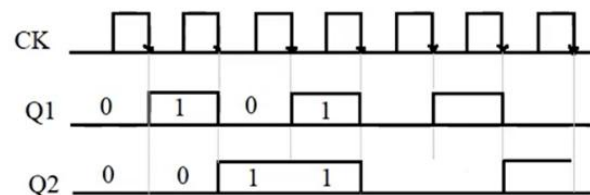
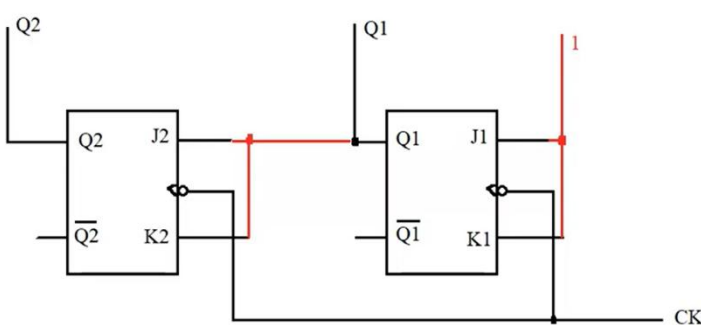
- **Contatori Asincroni:** Il clock arriva al primo FF e tutti gli altri hanno come clock le uscite del FF precedente
- **Contatori Sincroni:** Il clock arriva simultaneamente a tutti i FF

Contatore Asincrono MOD 4 (sensibile al fronte in discesa)

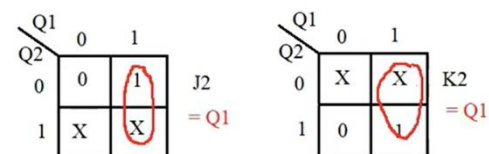


Il diagramma temporale appare così perché il FF di Q2 è sensibile al fronte in discesa di Q1, interpretato come segnale di Clock.

Contatore Sincrono MOD 4 (sensibile al fronte in discesa)



STATI	Q2	Q1	Q2 _{n+1}	Q1 _{n+1}	J2	K2	J1	K1
S0	0	0	0	1	0	X	1	X
S1	0	1	1	0	1	X	X	1
S2	1	0	1	1	X	0	1	X
S3	1	1	0	0	X	1	X	1



Possiamo dire che in questo caso l'esercizio risulta più complicato perché dobbiamo andare a svolgere un procedimento di Sintesi che ci permetta di ricavare il circuito combinatorio che collega un FF all'altro. In questo caso evidenziato in rosso

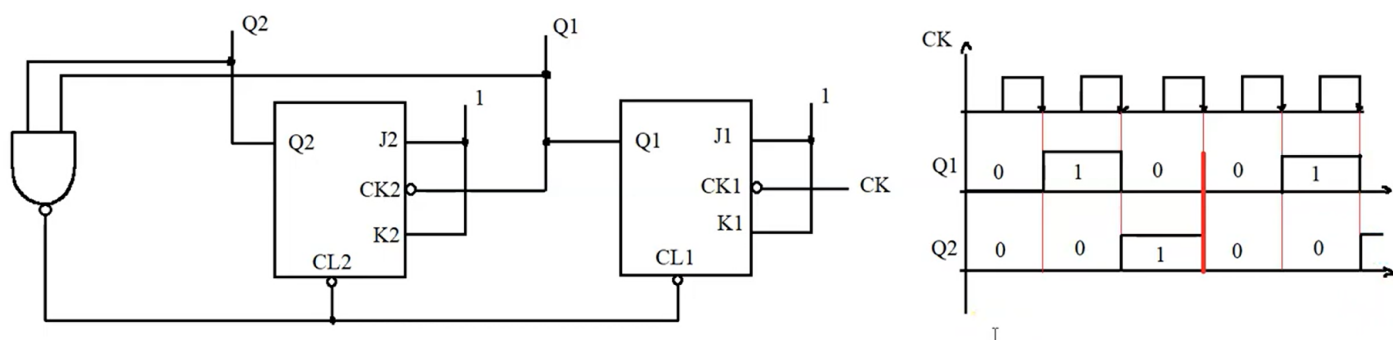
Realizzare Contatori MOD N (con N che NON è una potenza del 2)

Contatori Asincroni

Per i Contatori Asincroni possiamo semplicemente utilizzare il segnale di Clear, così da forzare il contatore a 0 una volta raggiunto il numero N a cui vogliamo contare.

Procedimento:

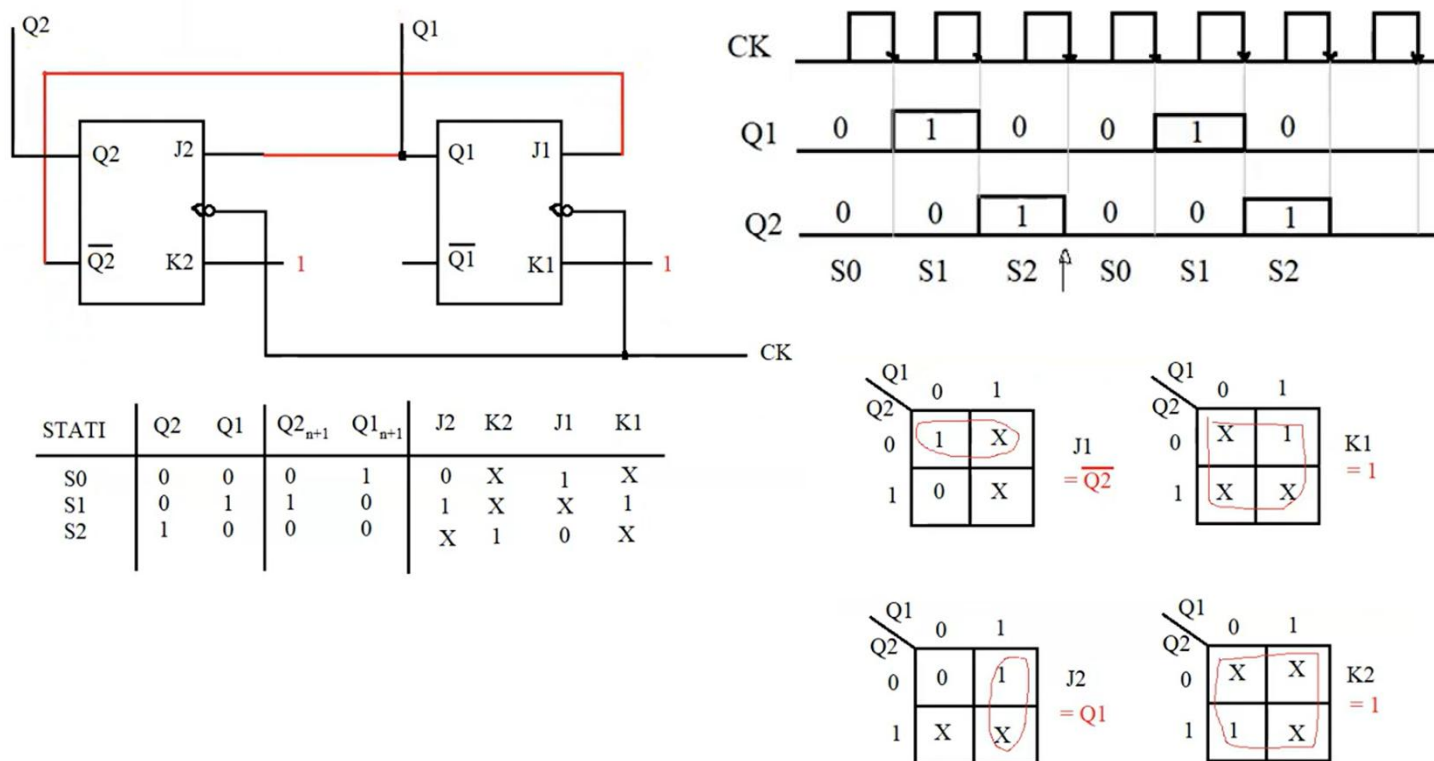
- 1) Iniziamo scegliendo una potenza del 2 che ci consenta di contare fino ad un numero maggiore di N. Ora siamo in grado di stabilire di quanti FF avremo bisogno.
- 2) Prendere le uscite che sono a 1 quando gli stati formano il numero N a cui dobbiamo fermarci, e metterle in NAND tra di loro. Il risultato di tale NAND sarà il segnale CLEAR da inviare ai FF.



Questo sopra raffigurato è un Contatore Asincrono MODULO 3 che, come visto prima, una volta raggiunto il conteggio a 3 forza il reset asincrono dei FF riportando il conteggio a 0.

Contatori Sincroni

Svolgiamo come sempre un procedimento di Sintesi, allo scopo di realizzare di un Automa ad HOC per contare in MODULO N.



Realizzare Contatori alla Rovescia (DOWN COUNTER)

I Contatori alla Rovescia (DOWN COUNTER) banalmente sono quelli che contano da N fino a 0. Sono identici in tutto e per tutto ai contatori normali (UP COUNTER) solo che invece di utilizzare le Uscite Normali dei FF utilizzano le Uscite Negate.

Contatore UP/DOWN

Ora che siamo in grado di realizzare un contatore in avanti ed uno alla rovescia possiamo combinare le due cose per realizzare un unico contatore in grado di decidere se contare avanti o indietro. È facilmente realizzabile inserendo una linea di controllo ed in base a questa decidere la direzione in cui contare.

Se per esempio decidessimo di inserire la linea di controllo dir che se vale 1 conta in avanti o se vale 0 conta alla rovescia, stiamo sostanzialmente inserendo un multiplexer tra un FF e l'altro che usa come linea di controllo Dir.

- Se **Dir = 0**, dobbiamo contare alla rovescia quindi selezioniamo **l'uscita negata del FF**
- Se **Dir = 1** dobbiamo contare in avanti quindi selezioniamo **l'uscita normale del FF**

